NOPT042 Constraint programming: Tutorial 12 - Planning

In [1]:
```
%load_ext ipicat
```

```
Picat version 3.9#12
```

The planning problem

Abstractly, planning refers to a class of problems where we are given:

- an initial state,
- final states (a set of them, or perhaps a property that makes a state final),
- a set of possible actions (how they transforming a state to another state),

and our task is to find a sequence of actions transforming the inital state to the final state, possibly optimizing some objective, and satisfying some resource limits. This problem appears in many practical applicatios (e.g. logistics, robotics), as well as in logical puzzles (e.g. the 15 puzzle) or games (e.g. Sokoban).

On an abstract level, we are trying to find a path (from the initial state to some final state) in the state graph (aka `state transition diagram`). The state graphis not explicitly constructed, as it is typically huge.

Several (all?) of the exercises from the last tutorial (on Tabling) can be seen as planning problems, generally following this pseudocode (the book):

```
table (+,-,min)

path(S,Plan,Cost),final(S) =>
    Plan=[],Cost=0.

path(S,Plan,Cost) =>
    action(S,NextS,Action,ACost),
    path(NextS,Plan1,Cost1),
    Plan = [Action|Plan1],
    Cost = ACost+Cost1.
```

The `planner` module

For planning problems, Picat provides the module `planner` which implements essentially the above pseudocode. It is enough to define the predicates

- `final(S)`,
- `action(S, NextS, Action, ACost)`,

and provide an initial state `SInit`. The `ACost` must be nonnegative. For example, if we only care about the number of steps, we set it to `1`. The actions are tried in the order in which they are defined (as rules defining the predicate `action`). Remember that if there are multiple actions, all but the last rule must be backtrackable (`?=>`).

The above pseudocode implements *depth-unbounded search*. The module `planner` also implements *depth-bounded search* (e.g. *iterative deepening* and *branch and bound*) and heuristic search.

See the guide, Chapter 8 for more details. (The module is not big, only ~300 LOC.)

In [2]:
```
#!wget http://picat-lang.org/download/planner.pi
!cat planner.pi
```

```
 module planner.


% best_plan(S,Limit,Plan) => best_plan(S,Limit,Plan).

% best_plan(S,Limit,Plan,PlanCost) => best_plan(S,Limit,Plan,PlanCost).

% best_plan(S,Plan,PlanCost) => best_plan(S,Plan,PlanCost).

% best_plan(S,Plan) => best_plan(S,Plan).

% best_plan_bin(S,Limit,Plan) => best_plan_bin(S,Limit,Plan).

% best_plan_bin(S,Limit,Plan,PlanCost) => best_plan_bin(S,Limit,Plan,PlanCos
t).

% best_plan_bin(S,Plan,PlanCost) => best_plan_bin(S,Plan,PlanCost).

% best_plan_bin(S,Plan) => best_plan_bin(S,Plan).

% best_plan_bb(S,Limit,Plan) => best_plan_bb(S,Limit,Plan).

% best_plan_bb(S,Plan,PlanCost) => best_plan_bb(S,Plan,PlanCost).

% best_plan_bb(S,Limit,Plan,PlanCost) => best_plan_bb(S,Limit,Plan,PlanCos
t).

% best_plan_bb(S,Plan) => best_plan_bb(S,Plan).

% best_plan_nondet(S,Limit,Plan) => best_plan_nondet(S,Limit,Plan).

% best_plan_nondet(S,Plan,PlanCost) => best_plan_nondet(S,Plan,PlanCost).

% best_plan_nondet(S,Limit,Plan,PlanCost) => best_plan_nondet(S,Limit,Plan,P
lanCost).

% best_plan_nondet(S,Plan) => best_plan_nondet(S,Plan).

% best_plan_unbounded(S,Limit,Plan) => best_plan_unbounded(S,Limit,Plan).

% best_plan_unbounded(S,Plan,PlanCost) => best_plan_unbounded(S,Plan,PlanCos
t).

% best_plan_unbounded(S,Limit,Plan,PlanCost) => best_plan_unbounded(S,Limit,
Plan,PlanCost).

% best_plan_unbounded(S,Plan) => best_plan_unbounded(S,Plan).

% current_plan() = current_plan().

% current_resource() = current_resource().

% current_resource_plan(Amount,Plan) => current_resource_plan(Amount,Plan).

% current_resource_plan_cost(Amount,Plan,Cost) => current_resource_plan_cost
```

```
(Amount,Plan,Cost).

% insert_state_list(StateL,Elm) = insert_state_list(StateL,Elm).

% is_tabled_state(S) => is_tabled_state(S).

% new_state_list(List) = new_state_list(List).

% plan(S,Limit,Plan) => plan(S,Limit,Plan).

% plan(S,Plan,PlanCost) => plan(S,Plan,PlanCost).

% plan(S,Limit,Plan,PlanCost) => plan(S,Limit,Plan,PlanCost).

% plan(S,Plan) => plan(S,Plan).

% plan_unbounded(S,Limit,Plan) => plan_unbounded(S,Limit,Plan).

% plan_unbounded(S,Plan,PlanCost) => plan_unbounded(S,Plan,PlanCost).

% plan_unbounded(S,Limit,Plan,PlanCost) => plan_unbounded(S,Limit,Plan,PlanC
ost).

% plan_unbounded(S,Plan) => plan_unbounded(S,Plan).



% A state-list is an ordered list, where the ordering of symbols is determin
ed

% by the addresses of the symbols in the symbol table, not by the characters
that

% constitute the symbols. This ordering allows for efficient ordering of sym
bols.

% Note that this ordering is different from lexicographical ordering, which
is used by sort().



new_state_list(List) = SList => new_state_list_aux(List,[],SList).



new_stte_list_aux([],SList0,SList) => SList = SList0.

new_state_list_aux([E|List],SList0,SList) =>

    bp.b_INSERT_STATE_LIST_ccf(SList0,E,SList1),

    new_state_list_aux(List,SList1,SList).



insert_state_list(SList0,Elm) = SList => bp.b_INSERT_STATE_LIST_ccf(SList0,E
```

```
lm,SList).


%%%

current_resource() = Amount =>

    bp.b_PLANNER_CURR_RPC_fff(Amount,_Plan,_Cost).


current_plan() = Plan =>

    bp.b_PLANNER_CURR_RPC_fff(_Amount,Plan,_Cost).


current_resource_plan(Amount,Plan) =>

    bp.b_PLANNER_CURR_RPC_fff(Amount,Plan,_Cost).


current_resource_plan_cost(Amount,Plan,Cost) =>

    bp.b_PLANNER_CURR_RPC_fff(Amount,Plan,Cost).


%%%

plan(S,Plan),var(Plan) =>

    bp.picat_plan(S,268435455,Plan,_).   % the limit is assumed to be 268435
455

plan(_S,Plan) =>

    throw_plan_arg_error(1,Plan,_,plan).


plan(S,Limit,Plan),var(Plan),integer(Limit),Limit>=0 =>

    bp.picat_plan(S,Limit,Plan,_).

plan(S,Plan,PlanCost),var(Plan),var(PlanCost) =>

    bp.picat_plan(S,268435455,Plan,PlanCost).

plan(_S,Limit,Plan) =>

    throw_plan_arg_error(Limit,Plan,_,plan).
```

```
plan(S,Limit,Plan,PlanCost),var(Plan),var(PlanCost),integer(Limit),Limit>=0
=>

    bp.picat_plan(S,Limit,Plan,PlanCost).

plan(_S,Limit,Plan,PlanCost) =>

    throw_plan_arg_error(Limit,Plan,PlanCost,plan).



plan(S,Limit,Plan,PlanCost,FinS),var(Plan),var(PlanCost),integer(Limit),Limi
t>=0 =>

    bp.picat_plan(S,Limit,Plan,PlanCost,FinS).

plan(_S,Limit,Plan,PlanCost,_FinS) =>

    throw_plan_arg_error(Limit,Plan,PlanCost,plan).



%%%

plan_unbounded(S,Plan),var(Plan) =>

    bp.picat_plan_unbounded(S,268435455,Plan,_).   % the limit is assumed to
be 268435455

plan_unbounded(_S,Plan) =>

    throw_plan_arg_error(1,Plan,_,plan_unbounded).



plan_unbounded(S,Limit,Plan),var(Plan),integer(Limit),Limit>=0 =>

    bp.picat_plan_unbounded(S,Limit,Plan,_).

plan_unbounded(S,Plan,PlanCost),var(Plan),var(PlanCost) =>

    bp.picat_plan_unbounded(S,268435455,Plan,PlanCost).

plan_unbounded(_S,Limit,Plan) =>

    throw_plan_arg_error(Limit,Plan,_,plan_unbounded).



plan_unbounded(S,Limit,Plan,PlanCost),var(Plan),var(PlanCost),integer(Limi
t),Limit>=0 =>

    bp.picat_plan_unbounded(S,Limit,Plan,PlanCost).

plan_unbounded(_S,Limit,Plan,PlanCost) =>
```

```
    throw_plan_arg_error(Limit,Plan,PlanCost,plan_unbounded).




%%%

%%% iterative deepening

best_plan(S,Plan),var(Plan) =>

    best_plan_downward(S,0,268435455,Plan,_,_).

best_plan(_S,Plan) =>

    throw_plan_arg_error(1,Plan,_,best_plan).


best_plan(S,Limit,Plan),var(Plan),integer(Limit),Limit>=0 =>

    best_plan_downward(S,0,Limit,Plan,_,_).

best_plan(S,Plan,PlanCost),var(Plan),var(PlanCost) =>

    best_plan_downward(S,0,268435455,Plan,PlanCost,_).

best_plan(_S,Limit,Plan) =>

    throw_plan_arg_error(Limit,Plan,_,best_plan).


best_plan(S,Limit,Plan,PlanCost),var(Plan),var(PlanCost),integer(Limit),Limi
t>=0 =>

    best_plan_downward(S,0,Limit,Plan,PlanCost,_).

best_plan(_S,Limit,Plan,PlanCost) =>

    throw_plan_arg_error(Limit,Plan,PlanCost,best_plan).


best_plan(S,Limit,Plan,PlanCost,FinS),var(Plan),var(PlanCost),integer(Limi
t),Limit>=0 =>

    best_plan_downward(S,0,Limit,Plan,PlanCost,FinS).

best_plan(_S,Limit,Plan,PlanCost,_FinS) =>

    throw_plan_arg_error(Limit,Plan,PlanCost,best_plan).
```

```
%%%

best_plan_downward(S,Level,_Limit,_Plan,_PlanCost,_FinS) ?=>

    (bp.global_get('_$picat_log',0,1) -> printf("%% Searching with the bound
%d\n",Level); true),

    call_picat_plan(S,Level),

    fail.

best_plan_downward(S,_Level,_Limit,Plan,PlanCost,FinS),

    Map = get_table_map('_$planner'),

    Map.has_key($current_best_plan(S))

=>

    Map.get($current_best_plan(S)) = Plan,

    Map.get($current_best_plan_cost(S)) = PlanCost,

    Map.get($current_best_plan_fin_state(S)) = FinS.

best_plan_downward(S,Level,Limit,Plan,PlanCost,FinS) =>

    bp.global_get('_$planner_explored_depth',0,Depth),

    (Depth == 268435455 -> Level1 = Level+1; Level1 = Depth),

        Level1 =< Limit,

    best_plan_downward(S,Level1,Limit,Plan,PlanCost,FinS).


%%% iterative deeping and binary search

best_plan_bin(S,Plan),var(Plan) =>

    best_plan_downward_bin(S,0,268435455,Plan,_,_).

best_plan_bin(_S,Plan) =>

    throw_plan_arg_error(1,Plan,_,best_plan).


best_plan_bin(S,Limit,Plan),var(Plan),integer(Limit),Limit>=0 =>

    best_plan_downward_bin(S,0,Limit,Plan,_,_).

best_plan_bin(S,Plan,PlanCost),var(Plan),var(PlanCost) =>
```

```
    best_plan_downward_bin(S,0,268435455,Plan,PlanCost,_).

best_plan_bin(_S,Limit,Plan) =>

    throw_plan_arg_error(Limit,Plan,_,best_plan).



best_plan_bin(S,Limit,Plan,PlanCost),var(Plan),var(PlanCost),integer(Limit),
Limit>=0 =>

    best_plan_downward_bin(S,0,Limit,Plan,PlanCost,_).

best_plan_bin(_S,Limit,Plan,PlanCost) =>

    throw_plan_arg_error(Limit,Plan,PlanCost,best_plan).



best_plan_bin(S,Limit,Plan,PlanCost,FinS),var(Plan),var(PlanCost),integer(Li
mit),Limit>=0 =>

    best_plan_downward_bin(S,0,Limit,Plan,PlanCost,FinS).

best_plan_bin(_S,Limit,Plan,PlanCost,_FinS) =>

    throw_plan_arg_error(Limit,Plan,PlanCost,best_plan).



%%%

best_plan_downward_bin(S,Level,Limit,Plan,PlanCost,FinS) =>

    Map = get_table_map('_$planner'),

    loop_best_plan_downward_bin(S,Level,Limit,Plan,PlanCost,FinS,Map).



loop_best_plan_downward_bin(S,Level,Limit,_Plan,_PlanCost,_FinS,_Map) ?=>

    Level =< Limit,

    (bp.global_get('_$picat_log',0,1) -> printf("%% Searching with the bound
%d\n",Level); true),

    call_picat_plan(S,Level),

    fail.

loop_best_plan_downward_bin(S,_Level,_Limit,Plan,PlanCost,_FinS,Map),

    Map.has_key($current_best_plan(S))

=>
```

```
    Lower = Map.get($current_lower_bound(S),0),
    Upper = Map.get($current_best_plan_cost(S))-1,
    loop_best_plan_bin(S,Map,Lower,Upper,Plan,PlanCost).
loop_best_plan_downward_bin(S,Level,Limit,Plan,PlanCost,FinS,Map) =>
    bp.global_get('_$planner_explored_depth',0,Depth),
%    writeln(depth=Depth),
    (Depth == 268435455 -> Lower = Level+1; Lower = Depth),
    Map.put($current_lower_bound(S),Lower),
    Lower < Limit,
    Level1 = 2*Lower,
    NewLevel = cond(Level1>Limit, Limit, Level1),
    loop_best_plan_downward_bin(S,NewLevel,Limit,Plan,PlanCost,FinS,Map).

% binary search
loop_best_plan_bin(S,Map,Lower,Upper,Plan,PlanCost),
    Lower =< Upper
=>
    NewLimit = Lower + (Upper-Lower) div 2,
    (bp.global_get('_$picat_log',0,1) -> printf("%% Searching with the bound
%d\n",NewLimit); true),
    if call_picat_plan(S,NewLimit) then
        NewUpper = Map.get($current_best_plan_cost(S))-1,
        loop_best_plan_bin(S,Map,Lower,NewUpper,Plan,PlanCost)
    else
        NewLower = NewLimit+1,
        loop_best_plan_bin(S,Map,NewLower,Upper,Plan,PlanCost)
    end.
loop_best_plan_bin(S,Map,_Lower,_Upper,Plan,PlanCost) =>
```

```
    Map.has_key($current_best_plan(S)),

    Map.get($current_best_plan(S)) = Plan,

    Map.get($current_best_plan_cost(S)) = PlanCost.


%%%

best_plan_nondet(S,Plan),var(Plan) =>

    best_plan_nondet_aux(S,268435455,Plan,_).

best_plan_nondet(_S,Plan) =>

    throw_plan_arg_error(1,Plan,_,best_plan_nondet).


best_plan_nondet(S,Limit,Plan),var(Plan),integer(Limit),Limit>=0 =>

    best_plan_nondet_aux(S,Limit,Plan,_).

best_plan_nondet(S,Plan,PlanCost),var(Plan),var(PlanCost) =>

    best_plan_nondet_aux(S,268435455,Plan,PlanCost).

best_plan_nondet(_S,Limit,Plan) =>

    throw_plan_arg_error(Limit,Plan,_,best_plan_nondet).


best_plan_nondet(S,Limit,Plan,PlanCost),var(Plan),var(PlanCost),integer(Limi
t),Limit>=0 =>

    best_plan_nondet_aux(S,Limit,Plan,PlanCost).

best_plan_nondet(_S,Limit,Plan,PlanCost) =>

    throw_plan_arg_error(Limit,Plan,PlanCost,best_plan_nondet).


best_plan_nondet_aux(S,Limit,Plan,PlanCost) =>

    not not (M = get_global_map(),

             M.put('_first_best_plan',[]),

             best_plan_downward(S,0,Limit,Plan0,PlanCost0,_),    % use table
d search to find the first best plan

             M.put('_first_best_plan',(Plan0,PlanCost0))),
```

```
    get_global_map().get('_first_best_plan') = (Plan0,PlanCost),

    (    Plan = Plan0

    ;

        bp.picat_best_plan_nondet_nontabled_search(S,Plan,PlanCost),

        Plan0 != Plan

    ).


%%% Branch-and-Bound

best_plan_bb(S,Plan),var(Plan) =>

    loop_best_plan_bb(S,268435455,Plan,_).

best_plan_bb(_S,Plan) =>

    throw_plan_arg_error(1,Plan,_,best_plan_bb).


best_plan_bb(S,Limit,Plan),var(Plan),integer(Limit),Limit>=0 =>

    loop_best_plan_bb(S,Limit,Plan,_PlanCost).

best_plan_bb(S,Plan,PlanCost),var(Plan),var(PlanCost) =>

    loop_best_plan_bb(S,268435455,Plan,PlanCost).

best_plan_bb(_S,Limit,Plan) =>

    throw_plan_arg_error(Limit,Plan,_,best_plan_bb).


best_plan_bb(S,Limit,Plan,PlanCost),var(Plan),var(PlanCost),integer(Limit),L
imit>=0 =>

    loop_best_plan_bb(S,Limit,Plan,PlanCost).

best_plan_bb(_S,Limit,Plan,PlanCost) =>

    throw_plan_arg_error(Limit,Plan,PlanCost,best_plan_bb).


loop_best_plan_bb(S,Limit,Plan,PlanCost),

    (bp.global_get('_$picat_log',0,1) -> printf("%% Searching with the bound
%d\n",Limit); true),
```

```
    call_picat_plan(S,Limit)

=>

    get_table_map('_$planner').get($current_best_plan_cost(S)) = PlanCost1,

    loop_best_plan_bb(S,PlanCost1-1,Plan,PlanCost).

loop_best_plan_bb(S,_Limit,Plan,PlanCost) =>

    Map = get_table_map('_$planner'),

    Map.has_key($current_best_plan(S)),

    Map.get($current_best_plan(S)) = Plan,

    Map.get($current_best_plan_cost(S)) = PlanCost.



call_picat_plan(S,Limit) =>

    bp.global_set('_$planner_explored_depth',0,268435455),

    not not call_picat_plan_aux(S,Limit).  % discard exception catchers crea
ted by picat_plan



call_picat_plan_aux(S,Limit) =>

    bp.picat_plan(S,Limit,Plan,PlanCost,FinS),

    Map = get_table_map('_$planner'),

    Map.put($current_best_plan(S),Plan),

    Map.put($current_best_plan_cost(S),PlanCost),

    Map.put($current_best_plan_fin_state(S),FinS).



%%%

best_plan_unbounded(S,Plan),var(Plan) =>

    bp.picat_best_plan_unbounded(S,Plan,_).

best_plan_unbounded(_S,Plan) =>

    throw_plan_arg_error(1,Plan,_,best_plan_unbounded).



best_plan_unbounded(S,Limit,Plan),var(Plan),integer(Limit),Limit>=0 =>
```

```
    bp.picat_best_plan_unbounded(S,Plan,PlanCost),

    PlanCost=<Limit.

best_plan_unbounded(S,Plan,PlanCost),var(Plan),var(PlanCost) =>

    bp.picat_best_plan_unbounded(S,Plan,PlanCost).

best_plan_unbounded(_S,Limit,Plan) =>

    throw_plan_arg_error(Limit,Plan,_,best_plan_unbounded).


best_plan_unbounded(S,Limit,Plan,PlanCost),var(Plan),var(PlanCost),integer(L
imit),Limit>=0 =>

    bp.picat_best_plan_unbounded(S,Plan,PlanCost),

    PlanCost =< Limit.

best_plan_unbounded(_S,Limit,Plan,PlanCost) =>

    throw_plan_arg_error(Limit,Plan,PlanCost,best_plan_unbounded).


%%%

is_tabled_state(S) =>

    bp.b_IS_PLANNER_STATE_c(S).


throw_plan_arg_error(_Limit,Plan,_PlanCost,Source),nonvar(Plan) =>

    handle_exception($var_expected(Plan),Source).

throw_plan_arg_error(_Limit,_Plan,PlanCost,Source),nonvar(PlanCost) =>

    handle_exception($var_expected(PlanCost),Source).

throw_plan_arg_error(Limit,_Plan,_PlanCost,Source),integer(Limit) =>

    handle_exception($nonnegative_integer_expected(Limit),Source).

throw_plan_arg_error(Limit,_Plan,_PlanCost,Source) =>

    handle_exception($integer_expected(Limit),Source).
```

Depth-unbounded search

The module `planner` implements the following two predicates:

- `plan_unbounded(S, Limit, Plan, PlanCost)`: find any plan where `PlanCost <= Limit`
- `best_plan_unbounded(S, Limit, Plan, PlanCost)`: find the best plan

The arguments `PlanCost` and `Limit` can be omitted.

Resource-bounded search

The module `planner` also implements the following predicates:

- `plan(S, Limit, Plan, PlanCost)`: perform *resource-bounded search* (i.e., keep a resource amount, do not explore a state if the resource amount is negative or if the state has previously failed with the same or more resource), if the resource is plan length (number of actions), this is *depth-bounded search*.
- `best_plan(S, Limit, Plan, PlanCost)`: finds the lowest-cost plan, using *iterative deepening*; calls `plan/4` setting the initial cost to 0 and then iteratively increasing.
- `best_plan_bb(S, Limit, Plan, PlanCost)`: first find any plan using `plan/4`, then branch and bound lowering the limit.

And we can use the function `current resource() = Limit` which returns the resource of the last call of `plan`; this can be used to implement a heuristic to prune the search (e.g. in 01-knapsack, if taking all of the remaining items, ignoring weight, won't give us sufficient total value, better than best so far).

Example: 15 puzzle

We will use the `planner` module to solve the 15 puzzle. Before checking the solution, think about what are the states and actions.

See this paper for a solution and more examples.

In [3]:
```
!picat puzzle15/puzzle15.pi
```

```
right
right
down
left
up
left
down
down
left
up
right
down
down
right
right
up
left
down
left
left
up
right
up
right
down
right
up
up
left
down
left
left
up
```

In [4]:
```
!cat puzzle15/puzzle15.pi
```

```
% Adapted from Constraint Solving and Planning with Picat, Springer
% by Neng-Fa Zhou, Hakan Kjellerstrand, and Jonathan Fruhman
import planner.

main =>
    InitS = [(1,2),(2,2),(4,4),(1,3),
             (1,1),(3,2),(1,4),(2,4),
             (4,2),(3,1),(3,3),(2,3),
             (2,1),(4,1),(4,3),(3,4)],
    best_plan(InitS,Plan),
    foreach (Action in Plan)
       println(Action)
    end.

final(State) => State=[(1,1),(1,2),(1,3),(1,4),
                       (2,1),(2,2),(2,3),(2,4),
                       (3,1),(3,2),(3,3),(3,4),
                       (4,1),(4,2),(4,3),(4,4)].

action([P0@(R0,C0)|Tiles],NextS,Action,Cost) =>
    Cost = 1,
    (R1 = R0-1, R1 >= 1, C1 = C0, Action = up;
     R1 = R0+1, R1 =< 4, C1 = C0, Action = down;
     R1 = R0, C1 = C0-1, C1 >= 1, Action = left;
     R1 = R0, C1 = C0+1, C1 =< 4, Action = right),
    P1 = (R1,C1),
    slide(P0,P1,Tiles,NTiles),
    current_resource() > manhattan_dist(NTiles),
    NextS = [P1|NTiles].

% slide the tile at P1 to the empty square at P0
slide(P0,P1,[P1|Tiles],NTiles) =>
    NTiles = [P0|Tiles].
slide(P0,P1,[Tile|Tiles],NTiles) =>
    NTiles=[Tile|NTilesR],
    slide(P0,P1,Tiles,NTilesR).

manhattan_dist(Tiles) = Dist =>
    final([_|FTiles]),
    Dist = sum([abs(R-FR)+abs(C-FC) :
                {(R,C),(FR,FC)} in zip(Tiles,FTiles)]).
```

Exercise: 01-Knapsack

Implement the 01-knapsack problem using the `planner` module. (Every CSP can be viewed as a planning problem where states are partial assignments and actions represent the choice of value for a variable. We are looking for a path from the root of the search tree to one of the leaves.)

In [5]:
```
!cat knapsack/instance.pi
```

```
instance(ItemNames, Capacity, Values, Weights) =>
    ItemNames = {"tv", "desktop", "laptop", "tablet", "vase", "bottle", "jac
ket"},
    Capacity = 23,
    Values = {500,350,230,115,180,75,125},
    Weights = {15,11,5,1,7,3,4}.
```

In [6]:
```
!cd knapsack && picat knapsack.pi instance.pi
```

```
(take,tv)
(leave,desktop)
(take,laptop)
(take,tablet)
(leave,vase)
(leave,bottle)
(leave,jacket)
```

In [7]:
```
!cat knapsack/knapsack.pi
```

```
import planner.

main([Filename]) =>
    cl(Filename),
    instance(ItemNames, TotalCapacity, Values, Weights),
    AllItems = [(ItemNames[I], Values[I], Weights[I]) : I in 1..ItemNames.le
ngth],

    % state: S@(RemainingItems, RemainingCapacity)
    InitialState = (AllItems, TotalCapacity),

    % PlanCost is the value of items we did not take
    best_plan(InitialState, Plan, PlanCost),
    foreach (Action in Plan)
       println(Action)
    end.

% take the current item
action(CurrentState@(Items, Capacity), NextState, Action, Cost) ?=>
    Items = [Item | RemainingItems],
    Item = (ItemName, ItemValue, ItemWeight),
    Action = (take, ItemName),

    % taking an item costs nothing
    Cost = 0,

    % is this action valid?
    Capacity >= ItemWeight,

    % take the item, lower capacity
    NextState = (RemainingItems, Capacity - ItemWeight).


% leave the current item
action(CurrentState@(Items, Capacity), NextState, Action, Cost) =>
    Items = [Item | RemainingItems],
    Item = (ItemName, ItemValue, ItemWeight),
    Action = (leave, ItemName),

    % leaving an item costs its value
    Cost = ItemValue,

    % leave the item, capacity does not change
    NextState = (RemainingItems, Capacity).


% finish if no remaining items
final(S@(Items, Capacity)) => Items = [].
```

Exercise: Jugs

Solve the Three Jugs Problem (exercise 3.12/8 in the book):

> There are 3 water jugs. The first jug can hold 3 liters of water, the second jug can hold 5 liters, and the third jug is an 8-liter container that is full of water. At the start, the first and second jugs are empty. The goal is to get exactly 4 liters of water in one of the containers. (We are not allowed to spill water).

Generalize to any number of jugs with arbitrary maximum and intial volumes, and any target volume, e.g.:

```
picat jugs.pi "[3,5,8]" "[0,0,8]" 4
```